

Unit III

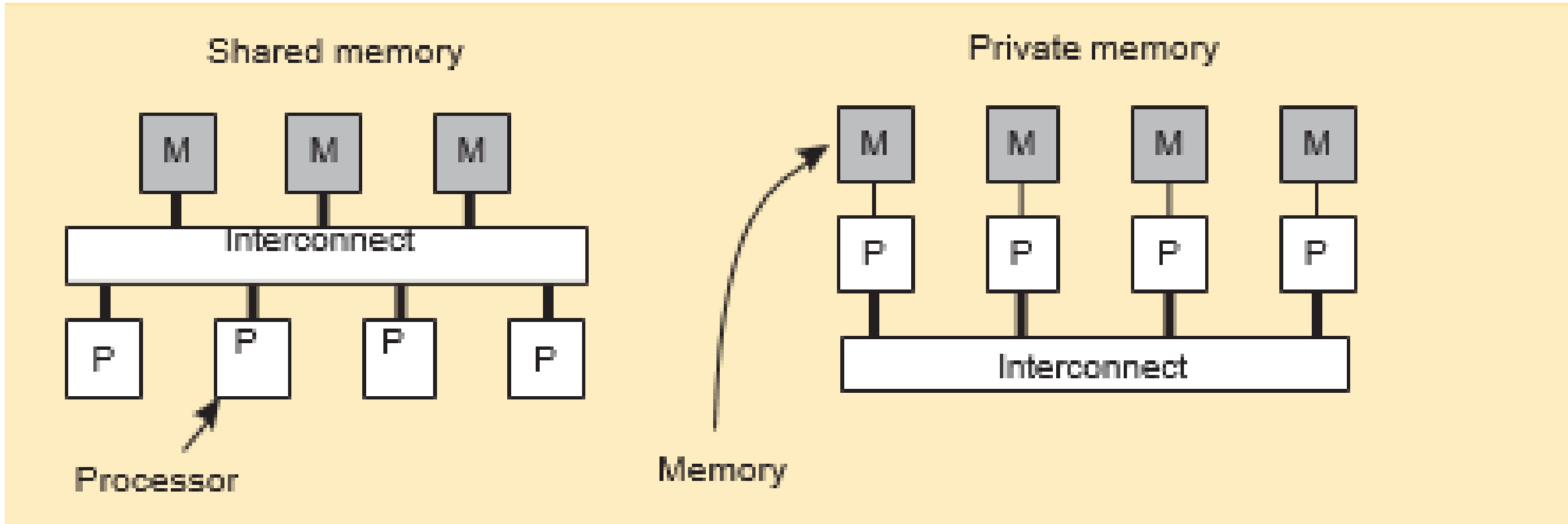
Three types of distributed systems

- High performance distributed computing systems
- Distributed information systems
- Distributed systems for pervasive computing

High performance distributed computing systems

Parallel Computing

- High-performance distributed computing started with parallel computing
- Multiprocessor and multicore versus multicomputer



Distributed shared memory systems

- Multiprocessors are relatively easy to program in comparison to multicomputers, yet have problems when increasing the number of processors (or cores). **Solution:** Try to implement a **shared-memory model** on top of a multicomputer.

Example through virtual-memory technique

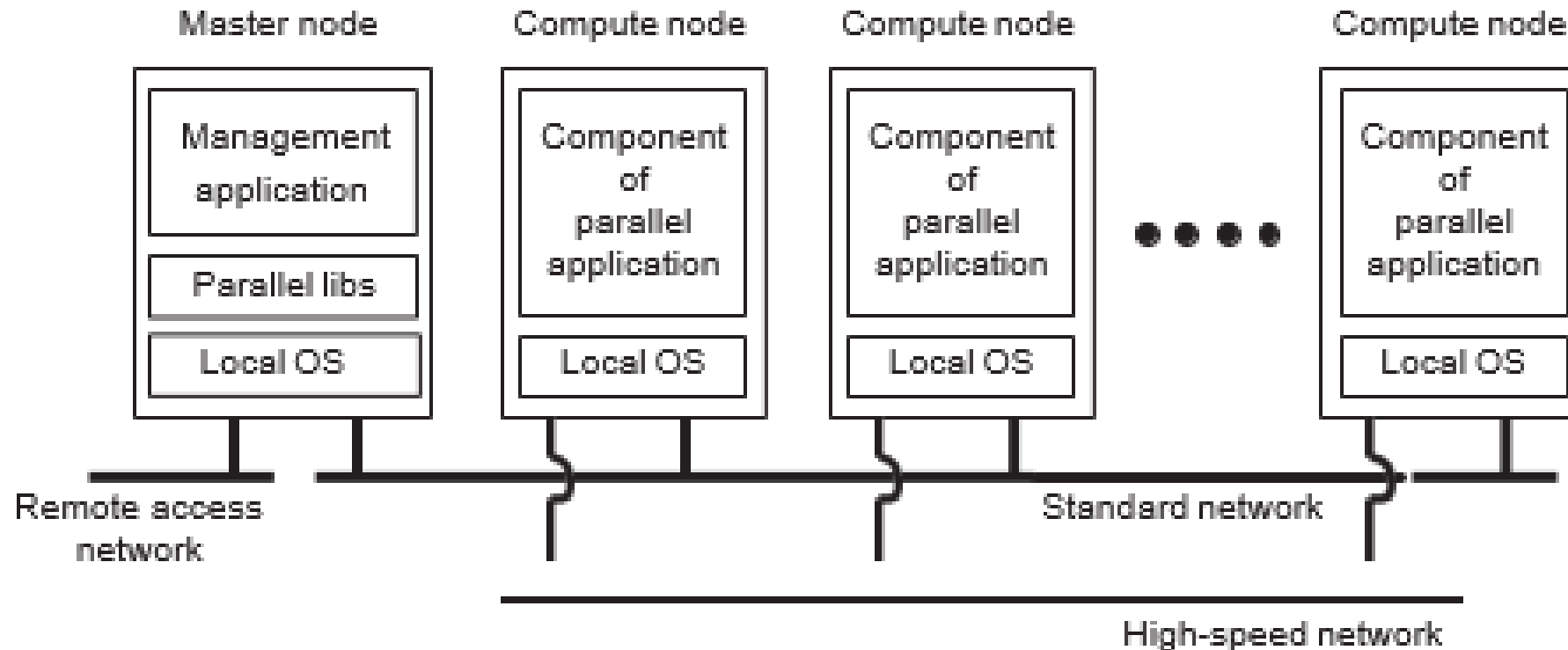
- Map all main-memory pages (from different processors) into one **single virtual address space**. If process at processor A addresses a page P located at processor B , the OS at A **traps and fetches P** from B , just as it would if P had been located on local disk.

Problem

- Performance of distributed shared memory could never compete with that of multiprocessors, and failed to meet the expectations of programmers. It has been widely abandoned by now.

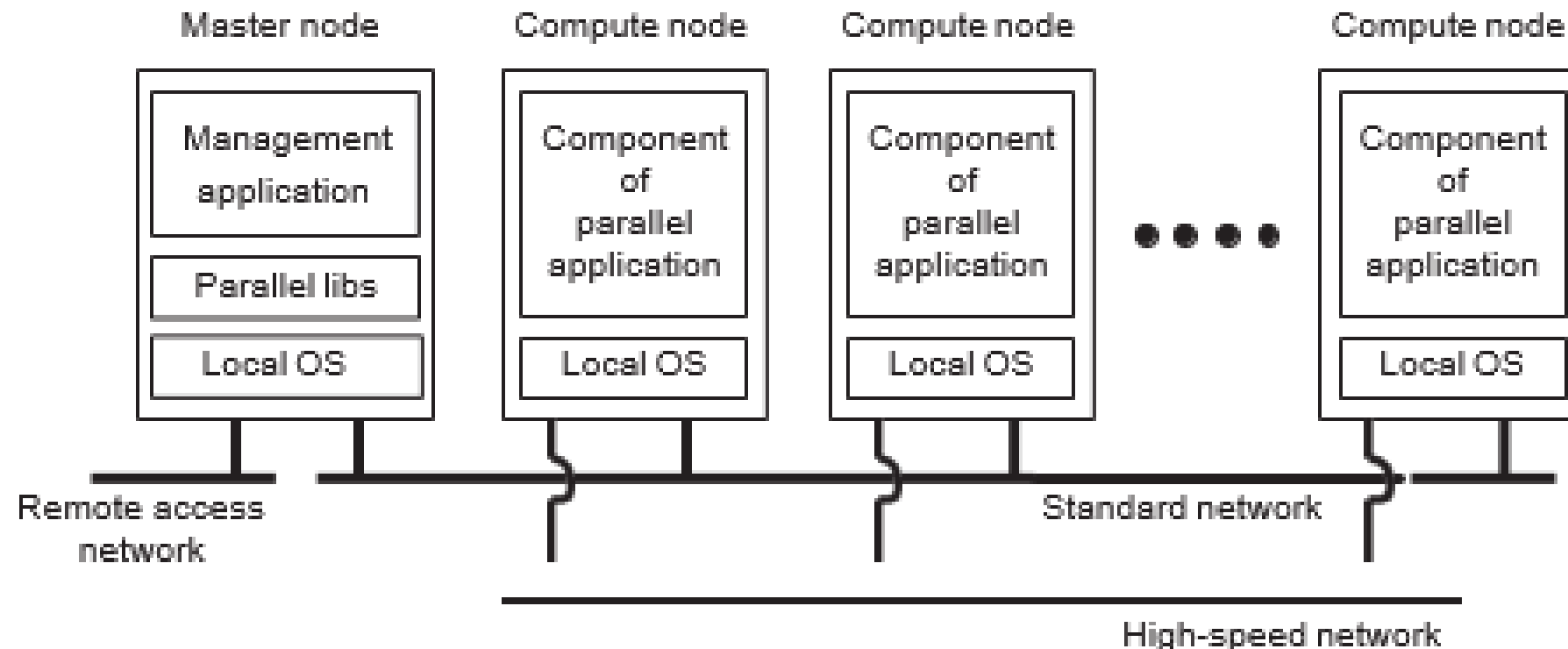
Cluster computing

- Essentially a group of high-end systems connected through a LAN
 - Homogeneous: same OS, near-identical hardware
 - Single managing node



Grid computing

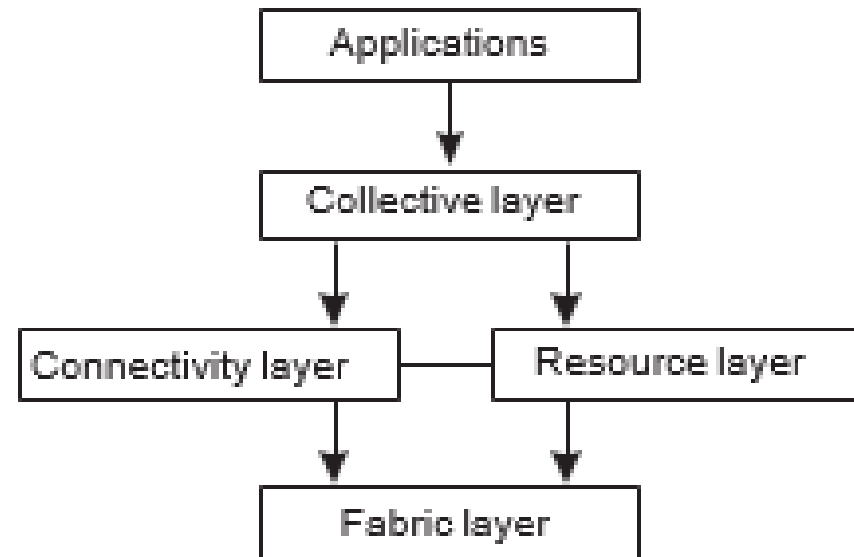
- **Heterogeneous:** No assumptions are made concerning similarity of hardware, os, networks, administrative domains, security policies etc.
- Resources from different organizations are brought together to allow the collaboration of a group of people from different institutions, indeed forming a federation of systems called **virtual organization**
- **Focuses** on providing access to resources from different administrative domains, and to only those users and applications that belong to a specific virtual organization



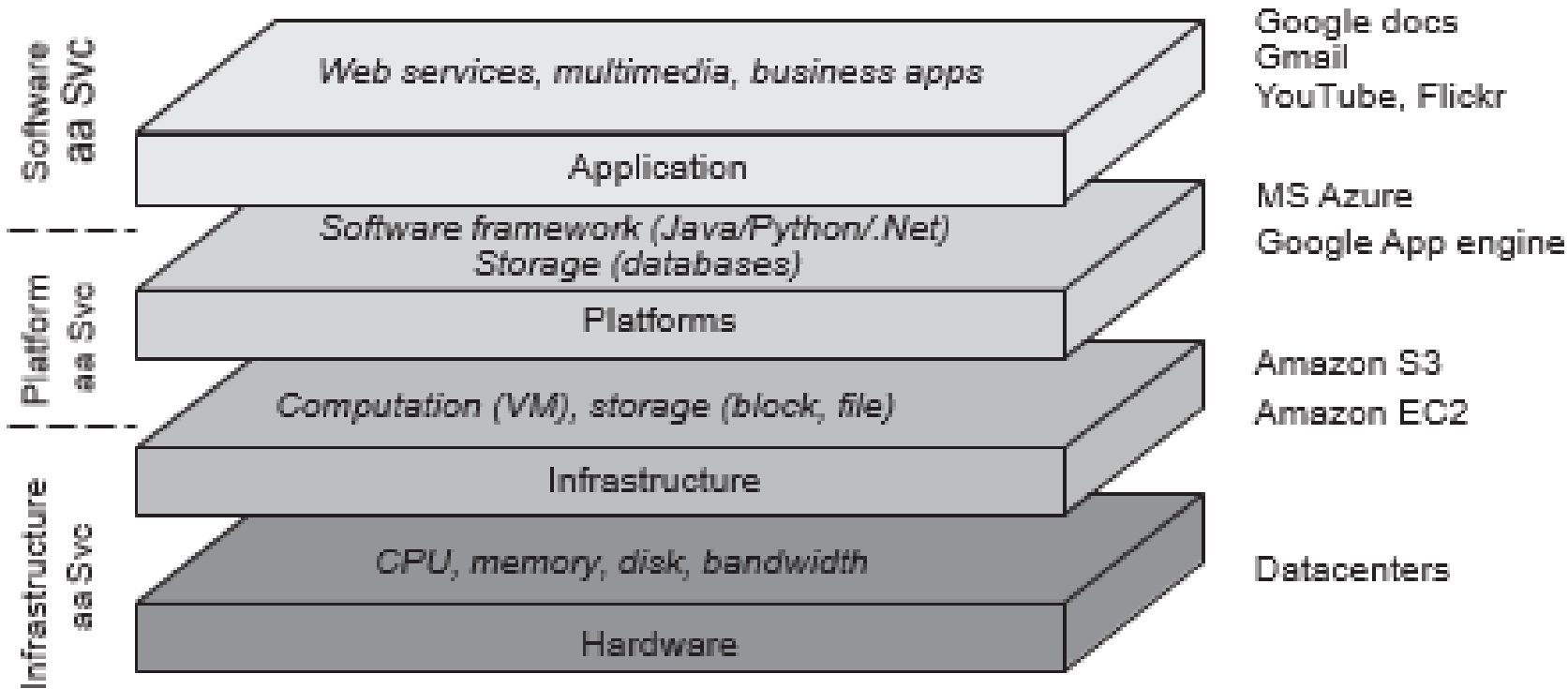
Architecture for grid computing

The layers:

- **Fabric:** Provides interfaces to local resources (for querying state and capabilities, locking, etc.)
- **Connectivity:** Communication/transaction protocols, e.g., for moving data between resources. Also various authentication protocols.
- **Resource:** Manages a single resource, such as creating processes or reading data.
- **Collective:** Handles access to multiple resources: discovery, scheduling, replication.
- **Application:** Contains actual grid applications in a single organization.



Cloud computing



Make a distinction between four layers

- **Hardware**: Processors, routers, power and cooling systems. Customers normally never get to see these.
- **Infrastructure**: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.
- **Platform**: Provides higher-level abstractions for storage and such. Example: Amazon S3 storage system offers an API for (locally created) files to be organized and stored in so-called **buckets**.
- **Application**: Actual applications, such as office suites (text processors, spreadsheet applications, presentation applications). Comparable to the suite of apps shipped with OSes.

Distributed Information Systems

Integrating applications

Situation

- Organizations confronted with many **networked applications**, but achieving interoperability was painful.

Basic approach

- A networked application is one that runs on a **server** making its services available to remote **clients**. Simple integration: clients combine requests for (different) applications; send that off; collect responses, and present a coherent result to the user.

Next step

- Allow direct application-to-application communication, leading to **Enterprise Application Integration**.

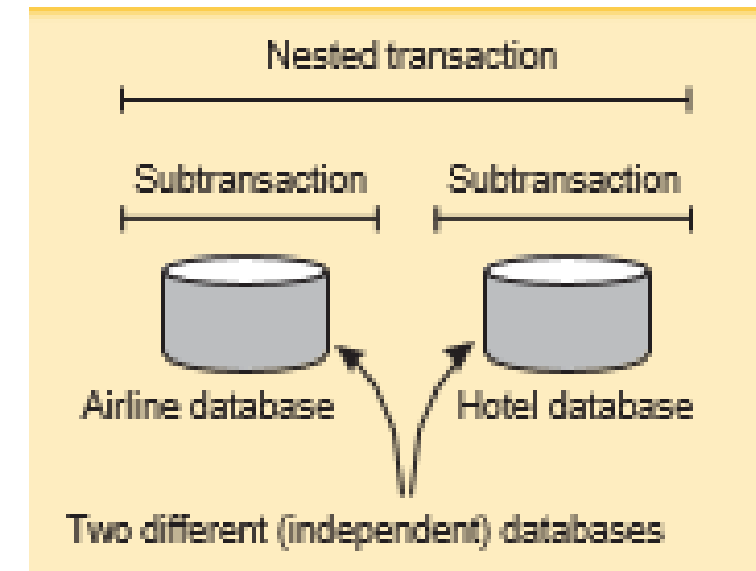
Example EAI: (nested) transactions

Transaction

Primitive	Description
<i>BEGIN TRANSACTION</i>	Mark the start of a transaction
<i>END TRANSACTION</i>	Terminate the transaction and try to commit
<i>ABORT TRANSACTION</i>	Kill the transaction and restore the old values
<i>READ</i>	Read data from a file, a table, or otherwise
<i>WRITE</i>	Write data to a file, a table, or otherwise

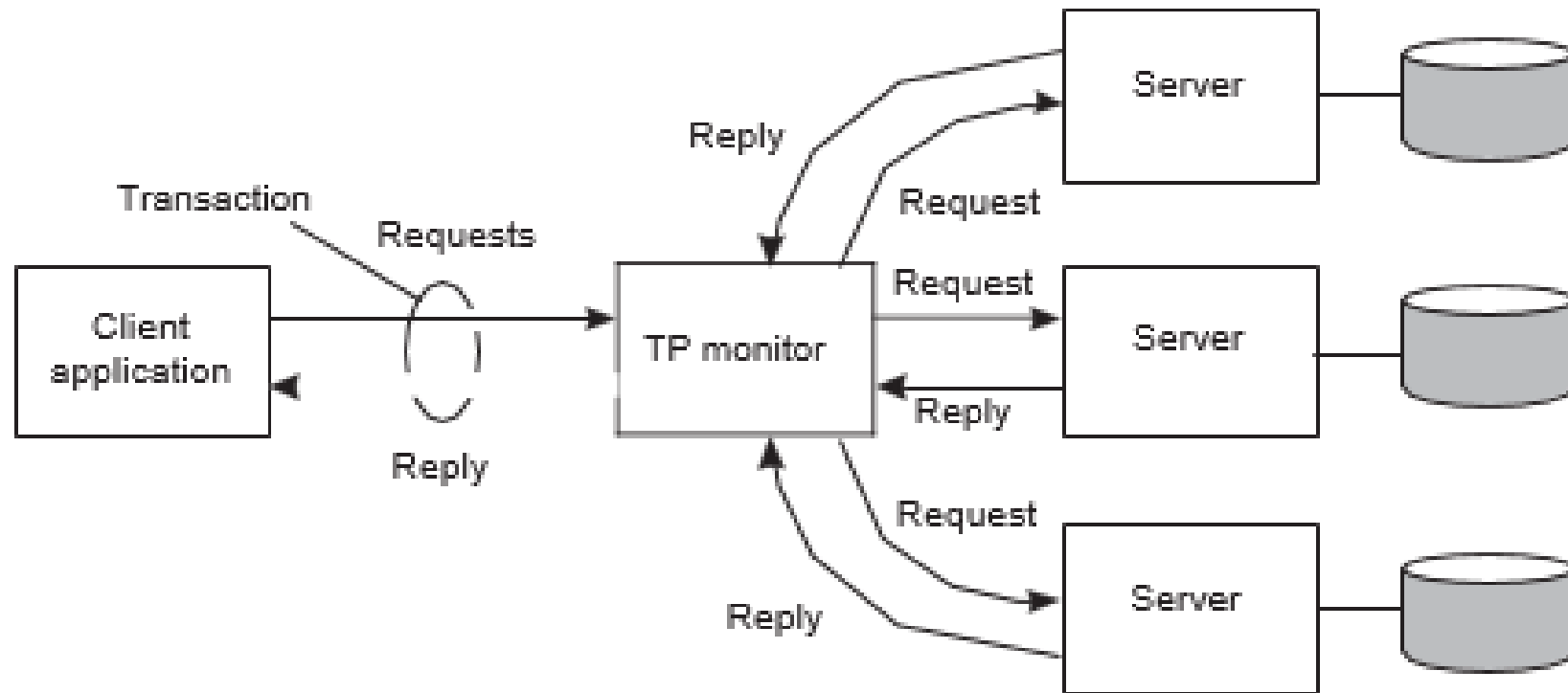
Issue: all-or-nothing

- **Atomic:** happens indivisibly (seemingly)
- **Consistent:** does not violate system invariants
- **Isolated:** not mutual interference
- **Durable:** commit means changes are permanent



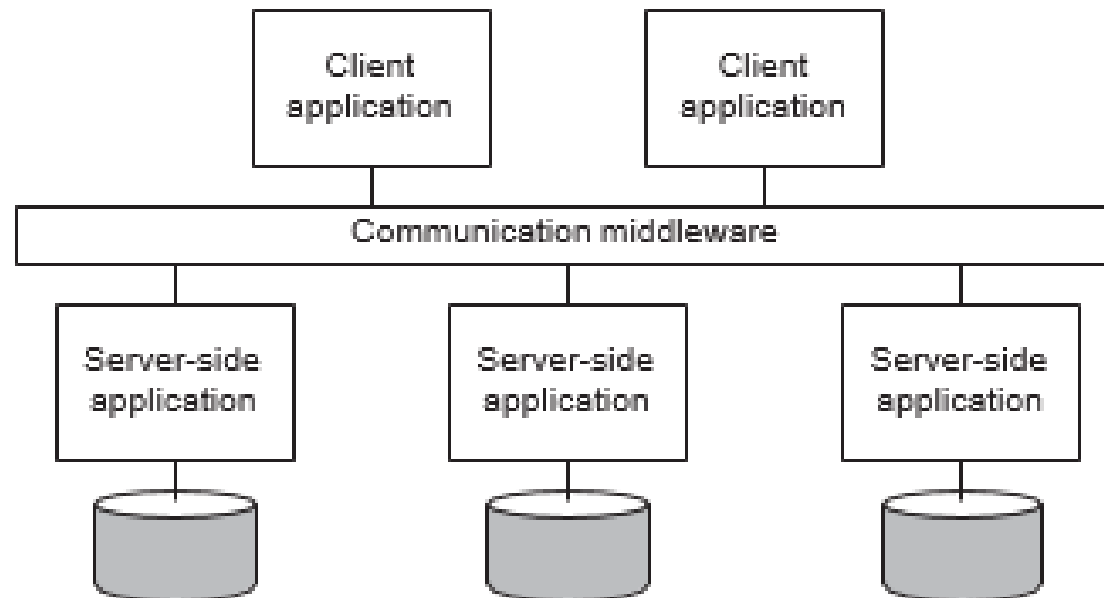
TPM: Transaction Processing Monitor

In many cases, the data involved in a transaction is distributed across several servers. A **TP Monitor** is responsible for coordinating the execution of a transaction.



Middleware offers communication facilities for integration

- **Remote Procedure Call (RPC):** Requests are sent through local procedure call, packaged as message, processed, responded through message, and result returned as return from call.
- **Message Oriented Middleware (MOM):** Messages are sent to logical contact point (**published**), and forwarded to **subscribed** applications.



How to integrate applications

- **File transfer:** Technically simple, but not flexible:
 - Figure out file format and layout
 - Figure out file management
- Update propagation, and update notifications.
- **Shared database:** Much more flexible, but still requires common data scheme next to risk of bottleneck.
- **Remote procedure call:** Effective when execution of a series of actions is needed.
- **Messaging:** RPCs require caller and callee to be up and running at the same time. Messaging allows decoupling in time and space.

- Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system, characterized by the fact that the system **naturally blends into the user's environment**.

Three (overlapping) subtypes

- **Ubiquitous computing systems**: pervasive and **continuously present**, i.e., there is a continuous interaction between system and user.
- **Mobile computing systems**: pervasive, but emphasis is on the fact that devices are **inherently mobile**.
- **Sensor (and actuator) networks**: pervasive, with emphasis on the actual (collaborative) **sensing** and **actuation** of the environment.

- **(Distribution)** Devices are networked, distributed, and accessible in a transparent manner
- **(Interaction)** Interaction between users and devices is highly unobtrusive **(Context awareness)** The system is aware of a user's context in order to optimize interaction
- **(Autonomy)** Devices operate autonomously without human intervention, and are thus highly self-managed
- **(Intelligence)** The system as a whole can handle a wide range of dynamic actions and interactions

Distinctive features

- A myriad of different mobile devices (smartphones, tablets, GPS devices, remote controls, active badges.
- Mobile implies that a device's location is expected to change over time $\Rightarrow$ change of local services, reachability, etc. Keyword: **discovery**.
- Communication may become more difficult: no stable route, but also perhaps no guaranteed connectivity $\Rightarrow$ **disruption-tolerant networking**.

Characteristics

- The **nodes** to which sensors are attached are:
- Many (10s-1000s)
- Simple (small memory/compute/communication capacity)
- Often battery-powered (or even battery-less)
- **Sensor networks as distributed databases**

